Tanuh AI cluster - User Guide - v1.0

1. Cluster description

The Tanuh AI cluster is a **Beowulf-class** cluster optimized for AI research. It utilizes a master-compute architecture where the master node handles management and storage, while specialized GPU nodes provide the raw compute power.

1.1. Hardware Overview

The Tanuh AI cluster today consists of 3 nodes , with 1 being master and 2 being the compute nodes. The master nodes only run CPUs to provide the base OS and co-ordination while compute nodes run both CPUs & GPUs. The master node also consists of a 100 TB shared data partition which is mounted on compute nodes via NFS,

Tier	**Node Name(s)**	**CPU Specs**	**Memory**	**GPU / Accelerator**
Master	master	2x AMD EPYC 9334 (64 cores total)	256 GB	CPU ONLY
Compute	n1	2x AMD EPYC 9534 (128 cores total)	1024 GB	4x **Nvidia RTX Pro 6000** (96GB VRAM)

Compute	n2	2x AMD EPYC 9534 (128 cores total)	1024 GB	8x **Nvidia H200** (141GB VRAM)

- **GPU Interconnect:** 200 Gb/s InfiniBand (HDR200) for high-speed internode data transfer.
- **Storage:** 132TB RAID 5 array partitioned into /home (User data/RAID) and /opt (Software stack).

1.2. Software Overview

The software stack deployed on the cluster can be thought of as layers , from the bare metal to the user environment. This is how they stack up.

Rocky Linux 8.10 (The OS)

The cluster is built on **Rocky Linux 8.10**, an enterprise-grade distribution that provides the core system services, hardware drivers, and network protocols.

It manages the computing resources in the cluster , data management , user management etc , ensuring your code can talk to the hardware.

While the OS includes a system-level Python (at /usr/bin/python3), this is reserved for system tasks and should **never** be used for your ML projects to avoid destabilizing the node.

Core Directories

- **/home**: This hosts each user's home directory. It is hosted on the master node and mounted on each compute node as an industry-standard **NFS filesystem**.
- **/opt/ohpc/pub/**: This directory is shared among all compute nodes. It contains cluster applications and libraries, making them appear as if they are installed locally on every node to maximize performance.

Cluster Framework - OpenHP v2.9

OpenHPC , sits on top of the OS and provides the curated set of tools and recipes necessary to manage the cluster as a single cohesive unit.

- It integrates critical infrastructure components, including Warewulf3 for stateful node provisioning and MUNGE for secure credential authentication between the master and compute nodes.
- It ensures a 1:1 environment mapping across the cluster, meaning a tool configured on the master node behaves exactly the same on the GPU compute nodes.

Workload Management (Slurm)

Slurm is the specialized engine responsible for resource management and job scheduling.It acts as the "traffic cop," taking your resource requests (e.g., "I need 2 GPUs and 64GB of RAM") and finding an available slot on the RTX Pro 6000 or H200 nodes.

You communicate with this layer using commands like sbatch to submit scripts or srun to launch interactive sessions.

The Module system (LMod)

Because the cluster supports various versions of compilers and libraries, software is not "installed" in the traditional sense. Instead, it is organized into **modules** managed by **LMod**.

LMod (The Manager): A tool that dynamically modifies your environment variables (like $PATH and $LD_LIBRARY_PATH) to make specific software visible to your session.

When you run module load, you are using a Lua-based script to pull a specific tool out of the shared /opt/ohpc/pub/ storage and into your current shell.

LMod eliminates the need for hard-coding paths in your scripts. By using `module load`, you ensure your session is always using the cluster-optimized version of a tool rather than a generic system default.

Conda Distribution - Miniforge3

We use the Miniforge3 as the specific distribution of the Conda package manager installed on this cluster.

It provides the core conda executable and is pre-configured to prioritize the conda-forge channel—the industry standard for up-to-date scientific and machine learning packages.

It lives in the shared **/opt/ohpc/pub/apps/** directory, meaning every user can access the same base binaries without duplicating disk space across the cluster,

User AI workspace - Conda Environments

This is the final layer where your actual development happens.

> Conda allows you to create isolated "sandboxes" or environments. An environment is a self-contained directory that holds a specific version of Python and all the libraries (like PyTorch, TensorFlow, or JAX) required for a project.
>
> Environments ensure reproducibility. You can have one project using Python 3.9 and another using Python 3.13 on the same node without them ever interacting or clashing.

2. Access & Environment

Login

Access the cluster by SSH to Master Node **10.16.63.40** ,

From inside the master nodes, you can SSH to the compute node by using their hostnames (n1, n2) . The Cluster uses a private local network which is local to the cluster.

Working with Data

The cluster uses a high-capacity **120TB RAID 5 array** for storage. While /home is for your private research, the /data directory is the engine for vertical-based collaboration.

Vertical-Specific Storage

All shared project data is located in /data (a symbolic link to the 81TB /home/data partition).

- **Access:** You are assigned to one of 6 vertical groups (e.g., team-mhealth, team-bcd). You can only access the directory belonging to your group.
- **Collaboration:** Folders are configured with **SetGID**, meaning any new file you create is automatically owned by the team group, allowing your teammates to see and edit your

work immediately.
- **Safety (Sticky Bit):** You can edit shared files, but the **Sticky Bit** ensures you **cannot delete or rename** a file created by a teammate. Only the original creator or root can remove data.

Performance: Local Scratch vs. NFS

- **NFS (/home and /data):** These are network-based. They are great for persistent storage but can be a bottleneck for heavy AI training.

- **Local NVMe (/tmp or /scratch):** Each GPU compute node has an **8TB high-speed NVMe drive**.

- **Best Practice:** At the start of a training job, copy your active dataset from /data to the local scratch space on the compute node. This provides maximum I/O performance for your GPU workloads.

Working with Modules

You interact with the software stack using the module command:

- **module avail**: Lists all software available for you to load.
- **module spider <name>**: A powerful search tool to find modules, even if they are hidden by a dependency.
- **module load <package>/<version>**: Injects the software into your current path (e.g., module load gnu12).
- **module list**: Shows what you currently have loaded.
- **module purge**: Clears your environment (highly recommended before starting a new project).

Managing Your Own Conda Environments

Because /home is shared via NFS, your Conda environments are available on every node.

1. **Initialization (One-time only):** Load the base module: module load miniconda (or anaconda).
   Warning: Do **not** run conda init. It modifies your .bashrc and can cause conflicts with Slurm modules.
2. **Creating a Private Environment:**
   Always specify a path or name. For AI workloads:
   Bash
   conda create -n my_ai_env python=3.10 pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia

Working with Slurm

Slurm is the "Gatekeeper" of the cluster. You should **never** run computations directly on the login node; doing so can destabilize the system for everyone.

Core Commands

Command	Purpose
sinfo	View the status of nodes (idle, alloc, or down) and partitions.
squeue	View the list of all running and pending jobs.
srun	Run a job interactively or launch a task within a script.
sbatch	Submit a batch job script to the queue.
scancel <jobid>	Stop a job that is currently running or waiting.

Requesting the Right GPU

Your cluster has two distinct GPU tiers. Specify the correct one in your #SBATCH script:

- **Node A (RTX Pro 6000):** Best for general-purpose training and visualization.
- **Node B (Nvidia H200):** Optimized for massive memory-intensive models (141GB VRAM).

Sample Job Script (submit.sh)

Bash

```
#!/bin/bash
#SBATCH --job-name=ai_train
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --gres=gpu:1        # Request 1 GPU
#SBATCH --partition=normal    # Use the production partition [cite: 241]
#SBATCH --time=24:00:00       # Max runtime

module load openmpi4/4.1.6    # Load OpenHPC modules [cite: 242]
module load miniconda
source activate my_env
#SBATCH --time=24:00:00  #SBATCH --time=24:00:00
srun python my_training_script.py
```

3. Cluster Do's & Dont's

Connectivity & Networking

- **DON'T attempt to access the internet from compute nodes:** Compute nodes are on a **private internal network** (192.168.1.0/24) and cannot reach the external world. Perform all software downloads and updates on the Master Node.
- **DON'T manually change hostnames:** The cluster hostnames are hardcoded into the **Warewulf3** and **Slurm** databases; changing them will break node communication and provisioning.

Computing & Resource Management

- **DO use Slurm for all computational tasks:** Always submit your workloads using sbatch for background jobs or srun for interactive sessions.
- **DON'T run jobs directly on the Master Node:** The master node (SMS) is the "brain" of the cluster, responsible for authentication, provisioning, and storage management. Heavy computation here can freeze the cluster for all users.
- **DO check node availability before submission:** Use the sinfo command to see if the **RTX Pro 6000** (Node A) or **Nvidia H200** (Node B) partitions are idle or alloc (allocated).
- **DO match your workload to the GPU tier:** Use Node A (96GB VRAM) for standard training and Node B (141GB VRAM) for large-scale memory-intensive models.
- **DO use interactive sessions for debugging:** If your code is failing, use srun --partition=normal --gres=gpu:1 --pty bash to troubleshoot in real-time on a compute node rather than guessing with batch scripts.

Storage & Data Handling

- **DO use /data/<vertical> for team collaboration:** Place shared datasets and project files in your assigned vertical folder (e.g., /data/mh , /data/oc , /data/bcd) to leverage the **SetGID** and **Sticky Bit** protection.
- **DO use /home/<user> for personal context:** Keep your private code, configuration files, and personal research notes in your home directory, which is shared via **NFS** across all nodes.
- **DO use Local Scratch for I/O performance:** Each compute node has an **8TB NVMe** local drive. For high-speed training, copy your active datasets from /data to the local /tmp or /scratch at the start of your job.
- **DON'T keep persistent data on compute node scratch space:** Local scratch is intended for temporary data only. These drives may be periodically cleaned, and data left there is not backed up or visible from other nodes.
- **DON'T attempt to delete teammates' data:** The **Sticky Bit** (mode 3770) on vertical folders prevents you from deleting or renaming files you do not own, even if you have write access to the folder.

Software & Environment (Modules & Conda)

- **DO use module load to access software:** The cluster uses **LMod**; always load the required compiler or MPI implementation before running your code.

- **DO use module purge at the start of scripts:** Include module purge as the first command in your Slurm scripts to ensure a clean environment and prevent library conflicts from previous sessions.
- **DON'T install software in system directories:** Never attempt to install packages in /usr/bin or /opt. Use **Conda** for Python libraries
- **DON'T run conda init:** Running conda init modifies your .bashrc and can cause significant path conflicts with **Slurm** and **OpenHPC** modules.
- **DO use source activate in Slurm scripts:** To use your Conda environments within a job, load the miniconda module and then use source activate /path/to/your/env.